

SecurePass

Java-Based Password Security Analyzer and Generator

Project Report

Submitted by

Praveen P B

Registration Number: 24BCY10026

Course: Programming in Java – Evaluated Project

(VITyarthi – Build Your Own Project)

Table of Contents

Table of Contents.....	2
1. Introduction	3
2. Problem Statement.....	3
3. Objectives	3
4. Scope	3
5. Target Users.....	3
6. Functional Requirements	4
FR-1: Password Analysis.....	4
FR-2: Security / Vulnerability Checks.....	4
FR-3: Secure Password Generation	4
FR-4: Security Report.....	4
7. Non-Functional Requirements.....	4
8. System Architecture	4
9. Workflow.....	5
10. Use Case Diagram	6
11. Class Diagram.....	7
12. Sequence Diagram.....	8
13. Component Diagram.....	9
14. Database/Storage Design.....	9
15. Design Decisions and Rationale	9
16. Java Concepts Used	10
17. Implementation Details.....	10
18. Password Analysis Algorithm	10
19. Entropy Calculation	11
20. Password Generation Algorithm.....	11
21. Security Considerations	11
22. Error Handling	11
23. Testing Approach	12
24. Test Cases and Results.....	12
25. Screenshots / Results	13
Screenshot Checklist.....	13
27. Challenges Faced.....	13
28. Learnings and Key Takeaways	13
29. Limitations.....	13
30. Future Enhancements.....	13
31. Conclusion.....	14
32. References	14

1. Introduction

Passwords remain the most common line of defense for protecting digital accounts, yet a large proportion of security breaches trace back to weak, reused, or predictable passwords. Most users have no simple, trustworthy way to check how strong a password actually is, or to generate a genuinely strong one, without pasting it into an online service of unknown trustworthiness.

SecurePass is a standalone Java desktop application that addresses this gap. It analyzes password strength entirely on the user's own machine — no password is ever transmitted, stored, or logged — and explains, in plain terms, why a password is weak or strong. It also generates cryptographically strong passwords using Java's SecureRandom facility. The project was built as the evaluated project for the Programming in Java course, and is designed to demonstrate object-oriented design, interfaces and polymorphism, collections, exception handling, file handling, and GUI programming in a single coherent, real-world application.

2. Problem Statement

Many users choose passwords that feel secure but are not — for example, passwords based on predictable substitutions ("P@ssw0rd"), keyboard patterns ("qwerty123"), or personal information. Online password-strength checkers require typing the password into a website, which is itself a security risk, and most give little explanation of their reasoning. There is a need for a lightweight, local, transparent tool that (a) evaluates a password against multiple concrete security criteria, (b) explains its verdict rather than issuing an opaque score, and (c) helps the user generate a strong replacement instantly, without any data ever leaving their computer.

3. Objectives

- To design and implement a Java desktop application that analyzes password strength using multiple, independently justifiable security criteria.
- To provide a transparent, explainable strength classification (Very Weak to Very Strong) rather than an unexplained numeric score.
- To implement a cryptographically sound password generator using `java.security.SecureRandom`.
- To ensure passwords are processed only in local memory and are never stored, logged, or transmitted.
- To demonstrate correct, purposeful use of core Java and object-oriented programming concepts covered in the syllabus.
- To produce a maintainable, modular, well-documented codebase suitable for academic evaluation and future extension.

4. Scope

In scope:

- Local, offline analysis of a single password entered by the user.
- Rule-based and entropy-based strength scoring with a small, bundled common-password sample list.
- Configurable secure password generation.
- Export of an analysis report that contains only derived metrics (never the password itself).

Out of scope:

- Checking passwords against large, live breached-password databases (e.g. Have I Been Pwned) — this would require network transmission of password data, which conflicts with the project's local-only security goal.
- Multi-user accounts, authentication, or network/server components.
- Mobile or web deployment — the project targets a single-user Windows/Linux/macOS desktop via Java Swing.

5. Target Users

- Students and individuals who want to understand what makes a password strong or weak.

- Anyone choosing a new password who wants a quick, trustworthy, offline strength check.
- Instructors/evaluators assessing the project for security-awareness and Java programming concepts.

6. Functional Requirements

FR-1: Password Analysis

The system shall accept a password (via a masked JPasswordField) and analyze its length, character composition, diversity, repetition, sequential patterns, and membership in a common-password list, then classify it into one of five strength levels with a plain-language explanation.

FR-2: Security / Vulnerability Checks

The system shall run a configurable set of independent checks (short length, repeated characters, sequential characters, common passwords, keyboard patterns, simple substitutions) and report which checks passed or failed, along with specific improvement recommendations.

FR-3: Secure Password Generation

The system shall generate a password of a user-specified length and character-class composition (uppercase, lowercase, digits, symbols) using SecureRandom, and shall immediately run the generated password through the same analyzer used for FR-1.

FR-4: Security Report

The system shall present a consolidated report (overall strength, entropy estimate, individual check results, recommendations) and shall optionally export this report to a local text file containing no plaintext password.

7. Non-Functional Requirements

ID	Requirement	Description
NFR-1	Security	No plaintext password is ever stored, logged, transmitted over a network, or written to an exported report.
NFR-2	Usability	The GUI is simple enough that a first-time user can analyze or generate a password without instructions.
NFR-3	Reliability	Invalid input (empty password, impossible generator settings, missing resource file) is handled gracefully without crashing the application.
NFR-4	Performance	Analysis and generation complete in well under 100ms for realistic password lengths, so the UI never appears to freeze.
NFR-5	Maintainability	Code is organized into clearly separated packages (model, analyzer, generator, security, gui, util) with single-responsibility classes and Javadoc comments.

8. System Architecture

SecurePass follows a simple layered architecture appropriate for a single-user desktop application. Each layer depends only on the layer(s) beneath it, which keeps the GUI free of business logic and keeps the analysis logic independently testable.

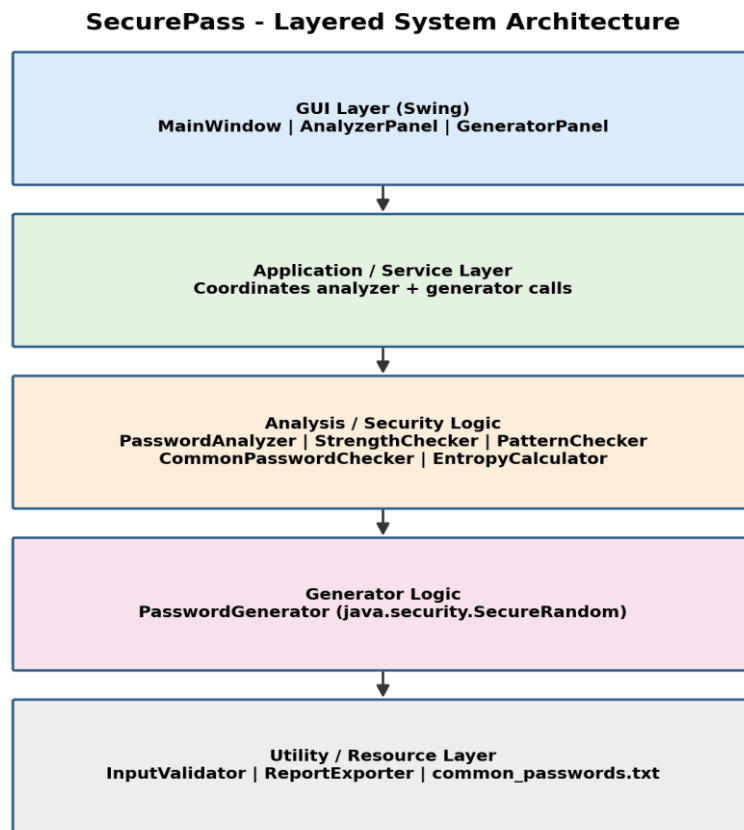


Figure 1: SecurePass layered system architecture

Data flow: the GUI layer captures the password as a `char[]` via `JPasswordField` and passes it to the service layer, which calls `InputValidator`, then `PasswordAnalyzer` (which runs every `SecurityCheck` and `EntropyCalculator`), and returns an immutable `PasswordResult` that the GUI renders. The `char[]` is cleared immediately after use rather than left in an immutable `String`, since Java `Strings` are not reliably erasable from memory.

9. Workflow

The diagram below shows the end-to-end workflow for the password analysis use case, from application launch through to optional report export.

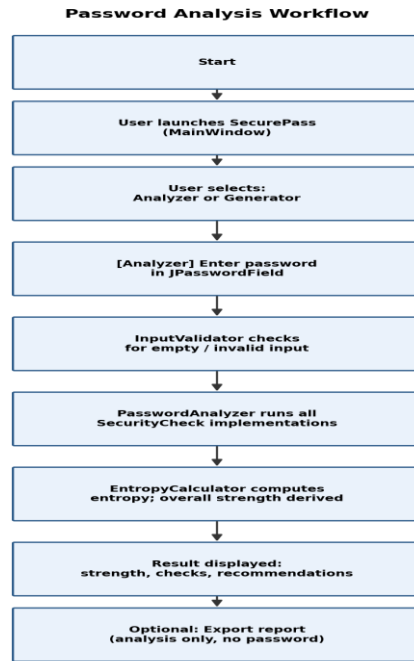


Figure 2: Password analysis workflow

10. Use Case Diagram

A single actor (the User) interacts with SecurePass through six use cases, covering both the analyzer and generator modules and the optional report export.

Use Case Diagram - SecurePass

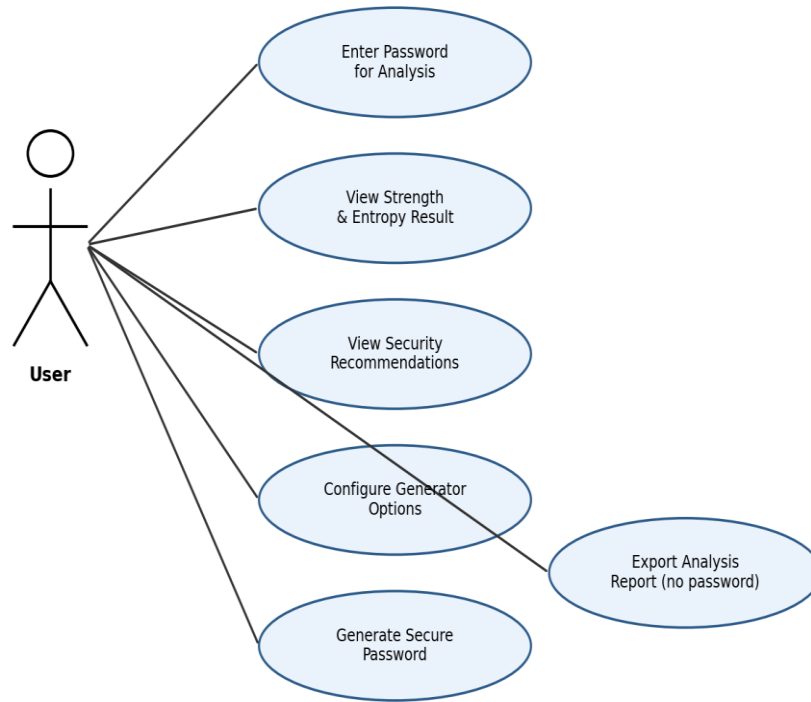


Figure 3: Use case diagram

11. Class Diagram

The class diagram below reflects the actual package structure of the implementation. The SecurityCheck interface is implemented by PatternChecker and CommonPasswordChecker, allowing PasswordAnalyzer to run a polymorphic list of checks without knowing their concrete types — this is the primary demonstration of interfaces and polymorphism in the project.

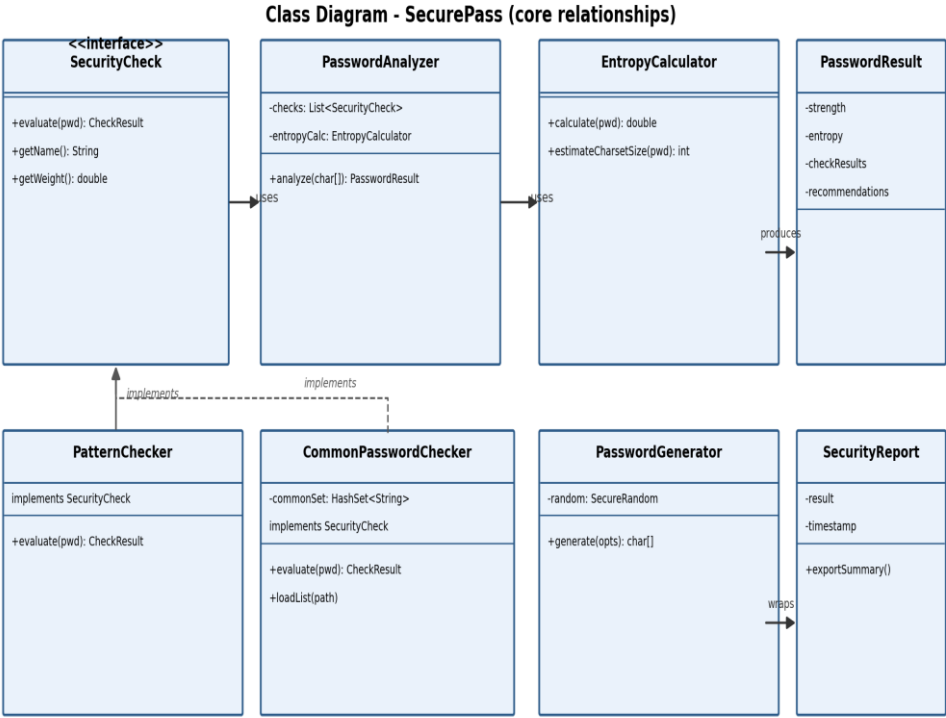


Figure 4: Class diagram (core relationships)

12. Sequence Diagram

The sequence diagram traces a single "Analyze Password" interaction, from user input through validation, analysis, entropy calculation, and final display.

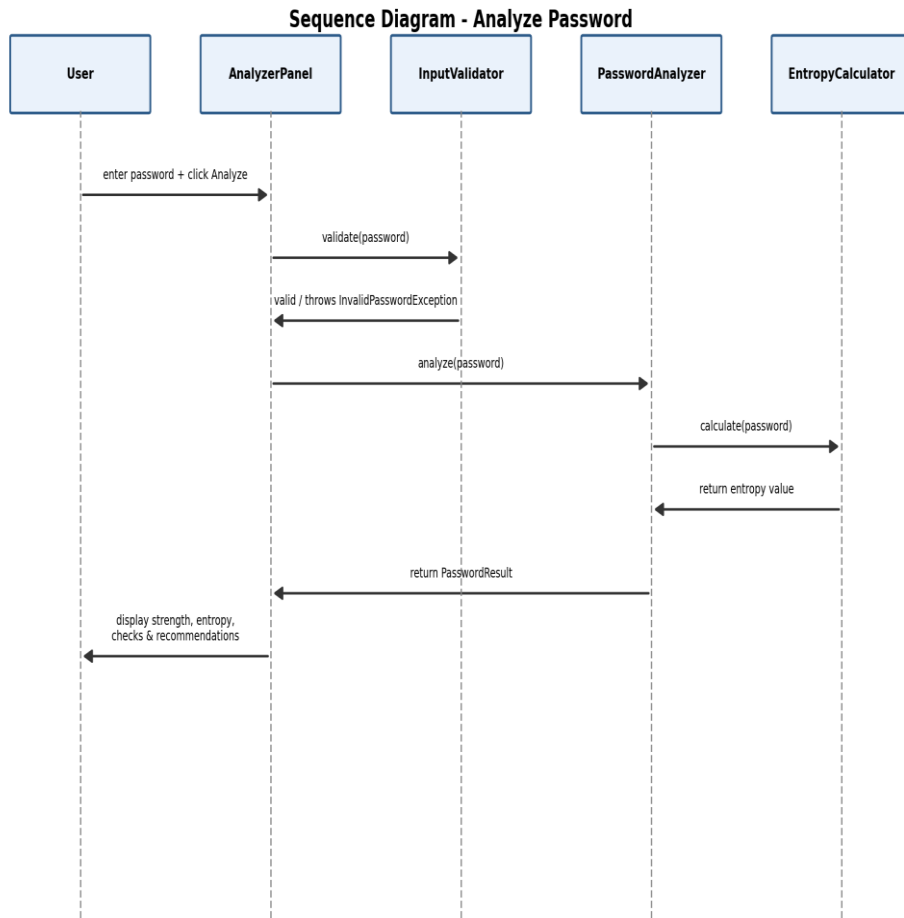


Figure 5: Sequence diagram – Analyze Password

13. Component Diagram

A separate component diagram is not included beyond the architecture and class diagrams above: at this project's scale, the five architectural layers already shown in Figure 1 and the class relationships in Figure 4 together fully describe how components are grouped and how they depend on one another. Introducing a sixth, largely duplicate diagram was judged to add clutter rather than clarity.

14. Database/Storage Design

SecurePass does not use a database. The only persistent data is a small, static, read-only educational word list, `common_passwords.txt`, loaded once at startup into a `HashSet<String>` for $O(1)$ lookup. Introducing a database (relational or otherwise) for a single, static, read-only resource would add complexity with no corresponding benefit, and was deliberately avoided — this decision is expanded on in Section 15. No ER diagram is therefore applicable.

15. Design Decisions and Rationale

Decision	Rationale
Interface-based SecurityCheck design	Allows new checks to be added later without modifying PasswordAnalyzer — a genuine, defensible use of polymorphism rather than an artificial one.
No database	The only persistent data is a static read-only word list; a HashSet loaded from a text file is simpler, faster, and equally correct for this scale.
No multithreading	Analysis and generation are simple, fast operations (well under the 100ms NFR target). Running them on a background thread would add

Decision	Rationale
	complexity (SwingWorker, synchronization) with no observable benefit, so it was deliberately left out rather than added artificially.
char[] instead of String for passwords	Java Strings are immutable and may remain in memory (e.g. in the string pool) until garbage collected; char[] can be explicitly zeroed immediately after use, which is the standard Java security practice for sensitive data.
Swing over JavaFX	Swing ships with the standard JDK, needs no extra runtime modules, and is more than sufficient for the simple forms/panels this project requires — keeping the project easy to run on any Windows laptop.

16. Java Concepts Used

Concept	Where demonstrated
Classes & Objects	Every module — PasswordAnalyzer, PasswordGenerator, PasswordResult, etc.
Encapsulation	PasswordResult and SecurityReport expose private fields via getters only; internal state cannot be mutated externally.
Interfaces & Abstraction	SecurityCheck interface hides each check's internal logic from PasswordAnalyzer.
Polymorphism	PasswordAnalyzer iterates a List<SecurityCheck>, calling evaluate() on each; the concrete behaviour differs per implementation.
Inheritance	GUI panels (AnalyzerPanel, GeneratorPanel) extend JPanel and override its lifecycle methods.
Collections (ArrayList, HashSet, HashMap)	ArrayList<SecurityCheck> as the check registry; HashSet<String> for O(1) common-password lookup; HashMap<String,Double> for per-check score breakdown.
String processing	Character classification, regex-based sequence/pattern detection in PatternChecker.
Exception handling	Custom InvalidPasswordException and GenerationException; try/catch around file I/O and user input in InputValidator.
File handling	Reading common_passwords.txt as a bundled resource; writing the exported analysis report.
Modular package structure	model, analyzer, generator, security, gui, util packages.
GUI programming	Java Swing: MainWindow, AnalyzerPanel, GeneratorPanel, JPasswordField.

Multithreading was deliberately not used — see the rationale in Section 15.

17. Implementation Details

The project is organized as a standard Java package hierarchy under securepass, split into six packages by responsibility:

- model — PasswordResult, SecurityReport (immutable data carriers)
- analyzer — PasswordAnalyzer, EntropyCalculator, PatternChecker, and the SecurityCheck interface
- security — CommonPasswordChecker
- generator — PasswordGenerator
- gui — MainWindow, AnalyzerPanel, GeneratorPanel
- util — InputValidator, ReportExporter

Class-by-class implementation notes, the compiled source listing, and build/run instructions are provided in the accompanying GitHub repository README.md.

18. Password Analysis Algorithm

Each SecurityCheck implementation returns a score contribution and a pass/fail verdict with an explanation. PasswordAnalyzer combines these into a single weighted score, which is then mapped to a strength category:

Factor	Weight (approx.)	Rationale
Length	30%	Length is the single strongest predictor of brute-force resistance.
Character diversity (upper/lower/digit/symbol)	25%	Increases the effective character-set size used in the entropy estimate.

Factor	Weight (approx.)	Rationale
Absence of repeated characters	15%	Repetition ("aaaa") sharply reduces effective randomness.
Absence of sequential patterns	15%	Sequences ("abcd", "1234") are among the first patterns attackers try.
Not in common-password list	15%	Common passwords are broken instantly regardless of apparent complexity.

The combined score (0–100) is mapped to five bands: Very Weak (0–20), Weak (21–40), Moderate (41–60), Strong (61–80), Very Strong (81–100). The exact thresholds are a reasonable, documented convention rather than a claim of scientific precision — see Section 21 (Limitations).

19. Entropy Calculation

SecurePass estimates Shannon-style password entropy using the standard approximation:

$$\text{Entropy} \approx L \times \log_2(R)$$

where L is the password length and R is the estimated size of the character set actually used (e.g. 26 for lowercase-only, 62 for mixed-case + digits, 94 for the full printable ASCII symbol set). `EntropyCalculator` determines R by inspecting which character classes are present, then computes bits of entropy. Edge cases handled: an empty password returns zero entropy rather than a mathematical error ($\log_2(0)$ is undefined), and a password using only one repeated character is still scored on the character-set size actually observed, not an assumed maximum.

Limitation, stated explicitly to the user in the GUI: this formula assumes each character was chosen independently and uniformly at random from R , which is not true of human-chosen passwords (e.g. "Password1!" scores well numerically despite being highly guessable). Entropy is shown as one input among several, not as a standalone guarantee of security.

20. Password Generation Algorithm

`PasswordGenerator` builds a character pool from the user-selected classes (uppercase, lowercase, digits, symbols), then uses `java.security.SecureRandom` — a cryptographically strong random number source, unlike `java.util.Random` — to pick each character independently. To guarantee that every selected class appears at least once, the generator seeds one random character per selected class first, then fills the remaining length from the full combined pool, and finally shuffles the result (Fisher–Yates, using `SecureRandom` for the swap indices) so the guaranteed characters are not predictably placed at the start.

Every generated password is immediately passed through the same `PasswordAnalyzer` used for user-entered passwords (FR-3), so the user sees its strength and entropy right away.

21. Security Considerations

- Passwords are captured via `JPasswordField` and handled as `char[]`, never as an immutable `String`, and are explicitly zeroed (`Arrays.fill(pwd, '\0')`) once analysis or generation is complete.
- No network calls are made anywhere in the application — all analysis is local.
- Exported reports (`ReportExporter`) contain only derived values (strength label, entropy, check results, recommendations) — the `ReportExporter` API does not accept a password parameter at all, which makes it structurally impossible to leak one into a report.
- `SecureRandom`, not `Math.random()` or `java.util.Random`, is used for all password generation.
- The application makes no claim that a password rated "Very Strong" is guaranteed to be uncrackable; this limitation is stated in the GUI itself.

22. Error Handling

`InputValidator` centralizes validation and throws specific checked exceptions that the GUI layer catches and turns into friendly dialog messages, rather than allowing the application to crash:

Condition	Handling
Empty password submitted for analysis	InvalidPasswordException → dialog: "Please enter a password to analyze."
Generator length outside allowed range	GenerationException → dialog: "Length must be between 4 and 128 characters."
No character class selected for generation	GenerationException → dialog: "Select at least one character type."
common_passwords.txt missing or unreadable	Caught IOException; CommonPasswordChecker falls back to an empty set and logs a non-fatal warning, so the rest of the analysis still runs.
Report export fails (e.g. disk full, no write permission)	Caught IOException → dialog with the underlying reason; the in-app report view is unaffected.

23. Testing Approach

Testing combines JUnit unit tests for the non-GUI logic (analyzer, entropy calculator, pattern checker, generator, validator) with manual exploratory testing of the Swing GUI, since GUI event flows are more practically verified by direct interaction. Unit tests focus on boundary and edge cases identified during design (empty input, single-character-class passwords, maximum-length input, missing resource file).

24. Test Cases and Results

⚠ This table lists the planned test cases as specified in the project brief. Actual/Status columns are marked "To be executed" and must be filled in with real results once the code is built and run — do not fill these in with invented outcomes.

Test ID	Input	Expected Output	Actual Output	Status
TC-01	Empty password submitted	Rejected with a validation message	To be executed	Pending
TC-02	Very short password (e.g. "ab")	Classified Very Weak with explanation	To be executed	Pending
TC-03	Lowercase-only password	Low diversity noted; strength capped accordingly	To be executed	Pending
TC-04	Mixed-case + digits + symbols password	Higher diversity score; higher strength band	To be executed	Pending
TC-05	Password with repeated characters ("aaaa1111")	Repetition check fails; flagged in recommendations	To be executed	Pending
TC-06	Password with sequential characters ("abcd1234")	Sequence check fails; flagged in recommendations	To be executed	Pending
TC-07	Common password (e.g. "password123")	Common-password check fails; rated Very Weak regardless of length	To be executed	Pending
TC-08	Strong generated password (16 chars, all classes)	Generator output passes all checks; rated Strong/Very Strong	To be executed	Pending
TC-09	Invalid generator length (e.g. 0 or 500)	GenerationException shown as a friendly dialog	To be executed	Pending
TC-10	No character category selected for generation	GenerationException shown as a friendly dialog	To be executed	Pending
TC-11	common_passwords.txt missing	Application continues; common-password check degrades gracefully	To be executed	Pending

TC-12	Report export	Report file created locally; contains no plaintext password	To be executed	Pending
-------	---------------	---	----------------	---------

25. Screenshots / Results

△ Screenshots are to be captured after the application is built and run, then inserted here. Section 26 below lists exactly which screenshots to take and the caption to use for each.

Screenshot Checklist

#	Screen / State	Suggested Caption
1	Main window on launch	Figure X: SecurePass main window
2	Password Analyzer – empty state	Figure X: Password Analyzer screen before input
3	Analyzer result – weak password	Figure X: Analysis result for a weak password, showing failed checks
4	Analyzer result – strong password	Figure X: Analysis result for a strong password
5	Recommendations panel	Figure X: Security recommendations for an improved password
6	Password Generator – options selected	Figure X: Generator configured with length and character types
7	Generated password + its analysis	Figure X: Generated password automatically analyzed
8	Validation error dialog	Figure X: Friendly error message for invalid input
9	Report export confirmation	Figure X: Exported analysis report (password-free)

27. Challenges Faced

△ To be completed once implementation is underway — record real difficulties encountered (e.g. tuning the scoring weights, Swing layout issues, regex edge cases in sequence detection) rather than generic statements.

28. Learnings and Key Takeaways

△ To be completed after finishing the project — reflect on what the interface-based SecurityCheck design taught you about polymorphism, what the `char[]` vs `String` decision taught you about Java memory handling, etc.

29. Limitations

- The entropy formula assumes uniform random character selection and does not capture human predictability (e.g. "Password1!" scores better than it deserves to).
- The bundled common-password list is a small educational sample, not a comprehensive, professionally maintained breach corpus.
- The scoring weights (Section 18) are a documented, defensible convention, not an empirically validated model.
- The tool cannot detect passwords that are predictable for reasons outside its checks (e.g. based on the user's own name, which it has no way to know).
- SecurePass never claims a password is guaranteed safe — only that it did or did not pass specific, named checks.

30. Future Enhancements

- Optional, privacy-preserving k-anonymity lookup against a breached-password API (e.g. Have I Been Pwned's range API, which never transmits the full password or hash).
- A larger, swappable common-password dataset loaded from an external, updatable file.
- Configurable scoring weights, exposed as an advanced settings panel.
- Passphrase generation mode (e.g. Diceware-style word-based passphrases) alongside character-based generation.
- Dark-mode theming and accessibility improvements to the Swing UI.

31. Conclusion

SecurePass meets the evaluated project's requirements for the Programming in Java course: it solves a genuine, well-scoped real-world problem; it is built from at least three (in fact four) functional modules across more than ten meaningful classes; it demonstrates interfaces, polymorphism, encapsulation, collections, exception handling, file I/O, and GUI programming, each for a real design reason rather than as decoration; and it is engineered with an explicit, defensible security posture — no password is ever stored, logged, or transmitted. The project is intended to be genuinely understandable and defensible in a viva, with every design decision traceable to a stated rationale.

32. References

1. Oracle, “java.security.SecureRandom”, Java SE Platform documentation.
2. Oracle, “Java Swing Tutorial”, The Java Tutorials.
3. C. Shannon, “A Mathematical Theory of Communication”, Bell System Technical Journal, 1948 (basis of the entropy estimate used).
4. NIST Special Publication 800-63B, “Digital Identity Guidelines: Authentication and Lifecycle Management” (guidance on password composition policy).
5. Have I Been Pwned, “Pwned Passwords API (k-Anonymity model)” — referenced as a future-enhancement design pattern, not used in the current implementation.